Week 4: K Shortest Paths & TSP Consolidation

Theory Guide — Hamiltonian Cycles & Travelling Salesman Problem

Module	Week 4 — Graph Algorithms
Topics	Graph Recap · Hamiltonian Cycle · TSP · Complexity
Prereqs	Basic graph theory, DFS/BFS, Dijkstra's algorithm
Level	Intermediate / Pre-competitive

1. Quick Graph Recap

Before diving into Hamiltonian cycles and TSP, make sure the following vocabulary is rock-solid. These ideas will appear constantly in competitive programming.

Term	Definition	Example
Vertex (node)	A point/city in the graph	City A, City B …
Edge	A connection between two vertices	Road from A to B
Weight	Cost/distance assigned to an edge	Distance = 42 km
Degree	Number of edges touching a vertex	City A has 3 roads
Path	A sequence of vertices connected by edges (no repeated vertex)	A → B → C → D
Cycle	A path that starts and ends at the same vertex	A → B → C → A
Complete graph K_n	Every pair of vertices is connected	K_4 has 6 edges
Connected graph	There is a path between every pair of vertices	All cities reachable

2. Hamiltonian Cycle

2.1 Formal Definition

A **Hamiltonian cycle** (also called a Hamiltonian circuit) in a graph G = (V, E) is a cycle that **visits every vertex exactly once** and returns to the starting vertex.

```
Definition — Hamiltonian Cycle:
Given a graph G with n vertices, a Hamiltonian cycle is a sequence
v0, v1, v2, ..., v(n-1), v0
such that:
(1) Each vi is a distinct vertex of G
(2) Each consecutive pair (vi, v(i+1)) is an edge in E
(3) The pair (v(n-1), v0) is also an edge in E (closing the cycle)
```

2.2 Hamiltonian Path vs. Hamiltonian Cycle

It is important to distinguish between a **Hamiltonian path** (visits every vertex exactly once, but does NOT need to return to start) and a **Hamiltonian cycle** (must return to start).

Property	Hamiltonian Path	Hamiltonian Cycle
Visits all vertices?	Yes	Yes
Returns to start?	No	Yes
Edges used	n - 1	n
Application	Route planning (one-way)	TSP, circular routes

2.3 Does a Hamiltonian Cycle Always Exist?

No! Not every graph has a Hamiltonian cycle. Deciding whether one exists is itself an NP-complete problem. However, some useful sufficient conditions exist:

Dirac's Theorem (1952): If every vertex in a simple graph with n >= 3 vertices has degree >= n/2, then the graph has a Hamiltonian cycle.

Ore's Theorem (1960): If for every pair of non-adjacent vertices u, v we have deg(u) + deg(v) >= n, then the graph has a Hamiltonian cycle.

Complete graphs K_n (n >= 3): Always have a Hamiltonian cycle.

Small example: *Consider K_4 (4 cities: A, B, C, D). One Hamiltonian cycle: A → B → C → D → A. Another valid one: A → C → B → D → A. (These are considered the same cycle traversed differently.)*

2.4 Counting Hamiltonian Cycles

For a complete graph K_n, the number of distinct Hamiltonian cycles is:

```
Number of Hamiltonian cycles in K_n = (n - 1)! / 2
```

For n = 4 cities: (4-1)!/2 = 6/2 = 3 cycles. For n = 10: 9!/2 = 181,440 cycles. For n = 20: 19!/2 ≈ 60 trillion cycles. This explosive growth is exactly why brute force doesn't scale!

3. The Travelling Salesman Problem (TSP)

3.1 Problem Statement

```
TSP — Formal Statement:
Given a complete weighted graph G = (V, E, w) where w(u,v) is the cost of edge (u,v),
find a Hamiltonian cycle of minimum total weight.

Input: n cities, pairwise distances d[i][j]
Output: A permutation of cities (a tour) that visits each city exactly once
and returns to start, minimising the total travel cost.
```

TSP is one of the most famous and extensively studied problems in computer science, operations research, and combinatorial optimisation. It has direct real-world applications in logistics, chip manufacturing, DNA sequencing, and more.

3.2 Variants of TSP

Variant	Key Constraint	Notes
Symmetric TSP	d[i][j] = d[j][i] for all i, j	Most common; undirected graph
Asymmetric TSP	d[i][j] may ≠ d[j][i]	Directed graph (one-way roads)
Metric TSP	Triangle inequality holds: d[i][k] ≤ d[i][j] + d[j][k]	Allows approximation guarantees
Euclidean TSP	Cities are points in 2-D plane; distance = straight-line	Special metric TSP

3.3 A Worked Example (n = 4)

Cities: 0 (Home), 1 (A), 2 (B), 3 (C). Distance matrix:

	0	1	2	3
0	**0**	**10**	**15**	**20**
1	**10**	**0**	**35**	**25**
2	**15**	**35**	**0**	**30**
3	**20**	**25**	**30**	**0**

All possible tours starting from city 0:

Tour	Cost Calculation	Total

0→1→2→3→0	10 + 35 + 30 + 20	95
0→1→3→2→0	10 + 25 + 30 + 15	80 ← OPTIMAL
0→2→1→3→0	15 + 35 + 25 + 20	95

4. Why is TSP Hard? NP-Hardness

4.1 Brute Force — The Naive Approach

The simplest algorithm tries every possible permutation of cities:

```
Algorithm — Brute Force TSP:
```

```
1. Fix starting city (say city 0) 2. Generate all permutations of remaining (n-1) cities 3. For each permutation, compute the tour cost 4. Return the permutation with minimum cost Time Complexity: O(n!) — catastrophically slow Space Complexity: O(n)
```

n (cities)	Permutations (n-1)!	Approx. time @ 10^9 ops/sec
5	24	< 1 microsecond
10	362,880	< 1 millisecond
15	87,178,291,200	~87 seconds
20	121,645,100,408,832,000	~3.8 million years
25	≈ 6.2 × 10^23	Longer than the universe's age

Key insight: Even doubling your CPU speed only adds ~1 extra city before hitting the same wall. The factorial growth completely dominates any hardware improvement. This is the fundamental reason TSP cannot be solved exactly for large inputs by brute force.

4.2 NP-Hardness — What Does It Mean?

TSP (decision version: 'Is there a tour of cost <= k?') is **NP-complete**. The optimisation version (find the minimum tour) is **NP-hard**. This means:

- No polynomial-time algorithm is known that solves all instances optimally
- It is believed (but not proven) that no such algorithm exists
- TSP is at least as hard as every other problem in NP
- This does NOT mean individual instances can't be solved fast — it means the worst case is exponential

4.3 Better Exact Algorithms

Algorithm	Time Complexity	Feasible up to ~n
Brute Force	`O(n!)`	`~12 cities`
Held-Karp DP (Bitmask)	`O(n^2 * 2^n)`	`~25 cities`
Branch & Bound	`Exponential (best case much better)`	`~40–50 cities (depends)`
Approximations (e.g. Christofides)	`O(n^3) — within 1.5x optimal`	`Millions of cities`

5. Held-Karp: Bitmask DP for TSP

The Held-Karp algorithm (1962) is the classic dynamic programming solution for small-to-medium TSP. It uses a bitmask to represent which cities have been visited.

5.1 State Definition

```
dp[mask][i] = minimum cost to reach city i, having visited exactly the
cities indicated by the bits in 'mask' (bit j is set if city j visited)

mask: an integer from 0 to 2^n - 1 (n bits, one per city)
i: the current city (last city visited)

Base case: dp[1][0] = 0 (start at city 0, only city 0 visited)
Answer: min over all i of { dp[(1<<n)-1][i] + dist[i][0] }
```

5.2 Recurrence

To fill dp[mask][i], consider all possible previous cities j (j is in mask and j != i):

```
dp[mask][i] = min over all valid j { dp[mask without bit i][j] +
dist[j][i] }
```

5.3 Complexity Analysis

- States: 2^n masks × n cities = n × 2^n states
- Transitions: For each state, we check up to n previous cities
- Total time: O(n^2 × 2^n) — much better than O(n!)
- Space: O(n × 2^n)

> *For n = 20: n! ≈ 2.4 × 10^18 vs n^2 × 2^n ≈ 4 × 10^8 operations. The DP is ~10 billion times faster for n = 20!*

6. K Shortest Paths

The **k shortest paths problem** asks: given source s and destination t, find the k paths with the lowest total cost (paths may repeat vertices).

6.1 Why Not Just Run Dijkstra k Times?

Dijkstra finds the single shortest path. Running it once gives you the 1st shortest path. But to get the 2nd, 3rd, ... kth, you cannot simply remove the first path — the paths are interleaved in

complex ways. You need a dedicated algorithm.

6.2 Yen's Algorithm (k Shortest Simple Paths)

Yen's algorithm finds k shortest *simple* paths (no repeated vertices). Idea:

1. Find the 1st shortest path using Dijkstra
2. For the p-th path, generate 'spur' paths by iteratively removing edges
3. Store candidate paths in a priority queue
4. Pop the minimum candidate as the next kth shortest path

Time complexity: O(k × n × (m + n log n)) where m = edges

6.3 Priority Queue (Heap) Approach

A simpler approach (allowing repeated vertices): use a modified Dijkstra with a count array — the t-th time city v is popped from the priority queue is its t-th shortest distance.

```
dp[v][k] = kth shortest distance to vertex v Time complexity: O(k × m × log(n × k))
```

7. Real-World Applications

Domain	TSP Application
Logistics & Delivery	UPS, FedEx route optimisation (saving millions in fuel per year)
Manufacturing	Drilling order for holes in printed circuit boards
DNA Sequencing	Reconstructing genome sequences from fragments
Astronomy	Optimal telescope pointing order for observations
Tourism	Planning road trips through multiple cities
Warehouse Robotics	Amazon warehouse picking robots

8. Summary & Key Takeaways

1. A Hamiltonian cycle visits every vertex exactly once and returns to start.
2. TSP = find the minimum-cost Hamiltonian cycle in a weighted complete graph.
3. Brute force TSP is O(n!) — completely infeasible beyond ~12 cities.
4. Held-Karp (bitmask DP) reduces this to O(n^2 × 2^n) — feasible up to ~25 cities.
5. TSP is NP-hard: no polynomial-time exact algorithm is known.

6. For large instances, approximation algorithms (like Christofides) are used in practice.

7. K shortest paths extends Dijkstra to find multiple shortest paths efficiently.

8. Real-world TSP instances with millions of cities are solved using heuristics and specialised solvers.

```
What to practise next:
- Implement Hamiltonian cycle detection using DFS + backtracking
- Code brute force TSP and benchmark it for n = 8, 10, 12
- Implement Held-Karp bitmask DP for TSP
- Try: Codeforces 8C (Looking for Order), SPOJ TSP problems, LeetCode
847
```